

System Design

Search as a Service

Technical Notes for Catalog Indexing and Search APIs

Document Scope

This document records the design decisions and implementation steps used to build a dedicated catalog Search Service. It covers the motivation for separating search from the Catalog Service, Elasticsearch fundamentals, asynchronous synchronization with RabbitMQ and MassTransit, client configuration, health checks, and the search endpoints implemented in the project.

Current implementation: catalog add/change/delete synchronization, multi-field search, filters, pagination, autocomplete, facets, highlighting, and liveness/readiness health checks.

Catalog Service (Publisher)

Functional decomposition means separating different responsibilities into independent services. If a service contains too many unrelated functions, decomposing them can improve **maintainability, independent deployment, fault isolation, and horizontal scalability**.

It also helps us follow the Single Responsibility Principle (SRP).

For example, imagine an e-commerce application where users need to search for products. The search results may support different operations such as:

- filtering by category or price
- sorting by newest products
- showing top-rated or top-selling items
- returning the top 10 results

Instead of putting all of this search logic inside the **Catalog Service**, it is usually better to separate it into a dedicated **Search Service**.

The **Catalog Service** can remain responsible for managing product data, while the **Search Service** is optimized for searching, filtering, and sorting products.

This separation allows the Search Service to be **developed, deployed, and scaled independently** from the Catalog Service.

Indexing in Relational Databases

Clustered Index: Sorts and stores the actual table rows physically based on the index key. A table can have only one clustered index.

Non-Clustered Index: Creates a separate structure containing indexed values and pointers to the actual rows. A table can have multiple non-clustered indexes.

Inverted Index: Maps terms to the documents that contain them. This structure is the foundation of efficient full-text search in engines such as Elasticsearch.

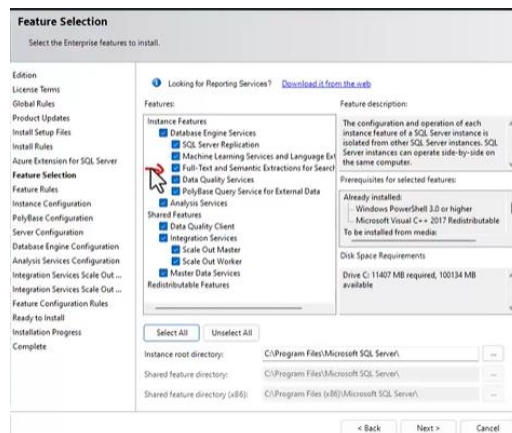


Figure 1. Full-Text Search feature selection during installation

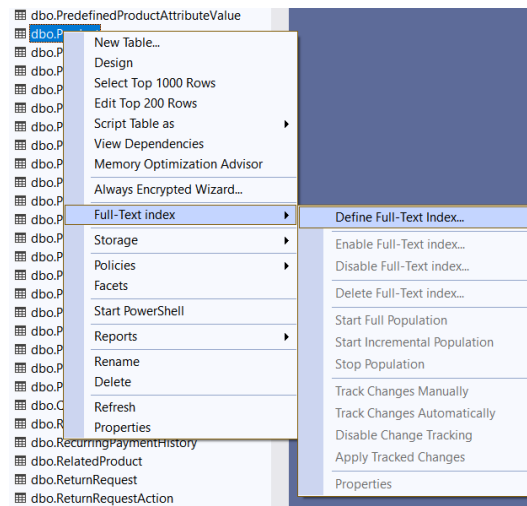


Figure 2. Full-text index definition

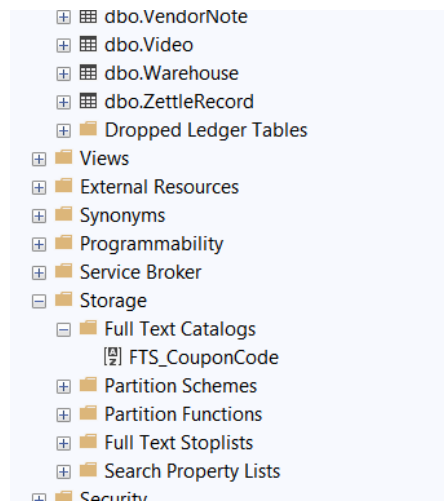


Figure 3. Full-text index in the Storage menu

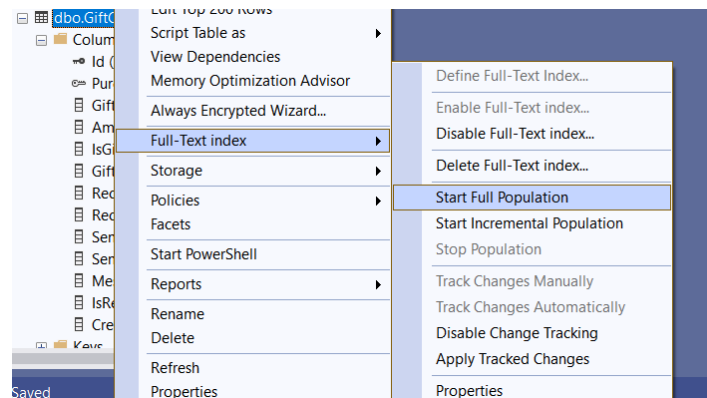


Figure 4. Starting to use the full-text index

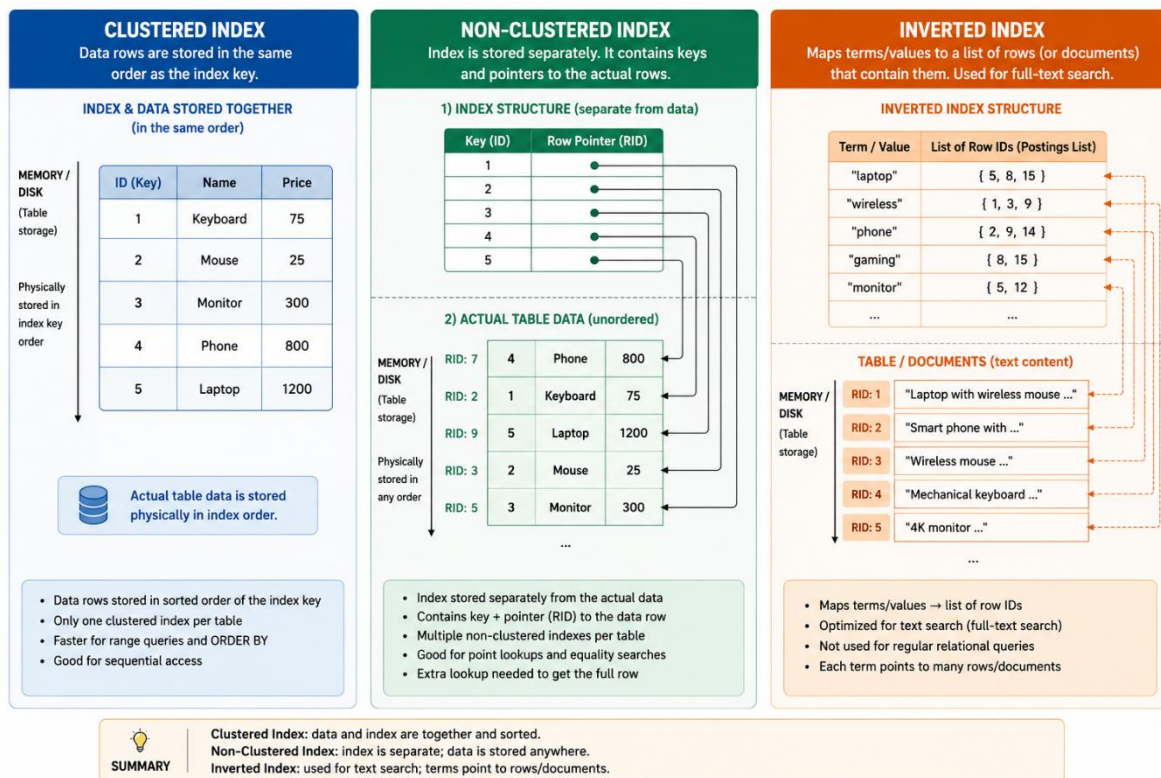
```

SELECT TOP (1000) [Id]
      , [Description]
FROM [Search].[dbo].[CatalogItem]
where FREETEXT(Description, 'Service')

```

Figure 5. Example full-text search syntax

The following diagram provides a high-level comparison of these three indexing approaches:



For text-heavy search scenarios, full-text indexing is generally more suitable than relying only on traditional clustered or non-clustered indexes.

Relational databases can support search, but complex full-text requirements may become harder to evolve as the searchable model grows. Adding searchable fields can require changes to indexes, queries, and the overall indexing strategy.

Because of this, engineers sometimes use a different database or search engine specifically for search operations.

For example, the Catalog Service may use a relational database as its primary source of truth, while the Search Service uses a separate data store optimized for searching, filtering, and indexing.

This approach introduces intentional data duplication because part of the catalog data is also stored in Elasticsearch. The relational database remains the source of truth, while Elasticsearch stores a search-optimized projection. For many systems, the benefits—better search capabilities, independent scalability, and flexible indexing—justify this duplication.

So the architecture may look like:

Catalog Service → Relational Database (source of truth)

Search Service → Elasticsearch (search-optimized copy)

The relational database remains the source of truth, while Elasticsearch contains a search-optimized copy of the catalog data.

Elasticsearch Structure

Elasticsearch is a **distributed search and analytics engine** built around documents rather than relational tables.

- **Cluster**
A cluster is a group of Elasticsearch nodes working together.
- **Node**
A node is a single Elasticsearch server that stores data and participates in searching and indexing.
- **Index**
An index is a logical collection of related documents. It is often compared with a table-like container in relational systems, although the concepts are not identical. Examples include products, orders, or customers.
- **Document**
Data is stored as JSON documents.
A document is roughly comparable to a row in a relational database.

```
{
  "id": 10,
  "name": "iPhone 16",
  "category": "Phone",
  "price": 900
}
```

- **Fields**
Each document contains fields such as name, price, and category.
- **Inverted Index**
Elasticsearch creates an inverted index for searchable text fields.

For example:

```
iphone → [Doc1, Doc5, Doc9]
phone  → [Doc1, Doc3, Doc5]
apple  → [Doc1, Doc8]
```

This makes text searching extremely fast.

- **Shard**
An index is divided into smaller pieces called **shards**.
Shards allow Elasticsearch to distribute data and search workload across multiple nodes.

Products Index

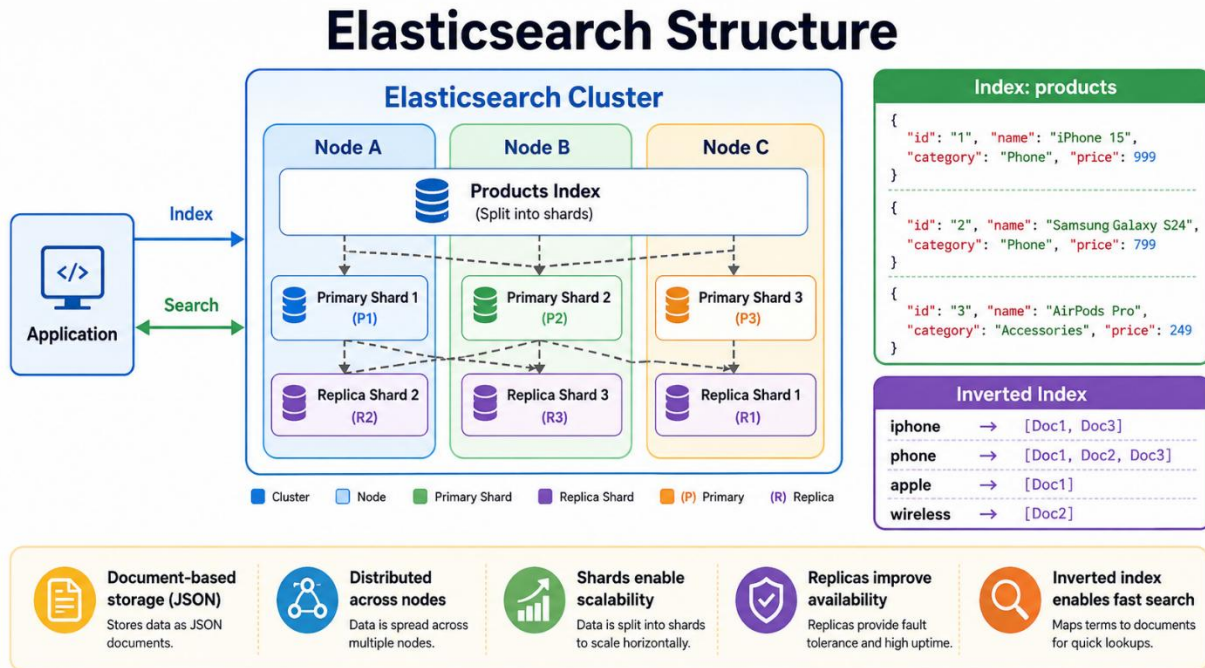
```
|
+-- Shard 1 → Node A
+-- Shard 2 → Node B
+-- Shard 3 → Node C
```

- **Replica**
Each shard can have one or more replicas.

```
Primary Shard → Node A
Replica Shard → Node B
```

Replicas provide **high availability** and can also improve search performance.

Elasticsearch Architecture Summary



The main idea is:

Elasticsearch stores JSON documents, builds inverted indexes for fast searching, and distributes the data across shards and nodes for horizontal scalability.

Installing Elasticsearch and Kibana with Docker Compose is very easy. You just need to download the `.env` and `docker-compose.yml` files from GitHub, then set the password and version values in the `.env` file.

After that, open a terminal in the folder containing these files and run:

```
docker compose up -d
```

Using this approach, we can start a multi-node Elasticsearch cluster together with Kibana. In the sample setup used by this project, the `es01` node is exposed on port 9200.`

In the next step, we need to synchronize the data from our relational database with Elasticsearch so that it can be used by the search service.

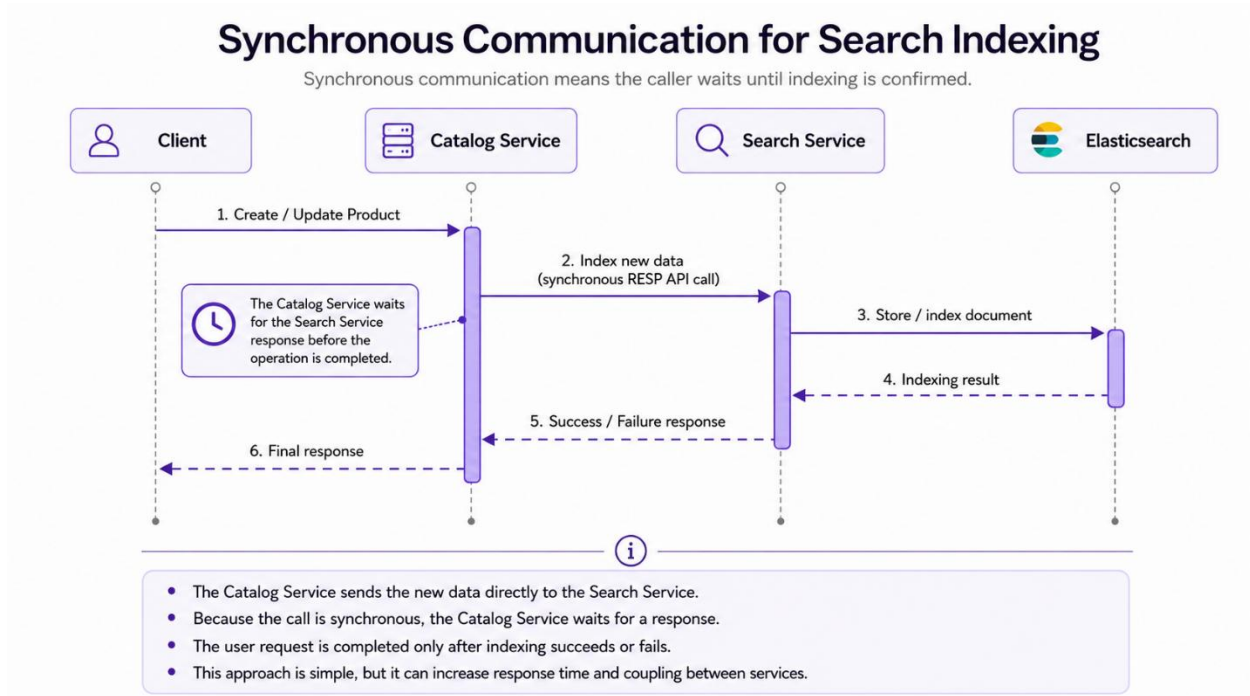
To achieve this, we need a mechanism that reacts whenever catalog data is added, updated, or deleted and keeps the Elasticsearch index synchronized with those changes.

We generally have two options:

- REST API (synchronous communication)
- Messaging (asynchronous communication)

What are the differences between these two approaches, and which one should we choose for this scenario?

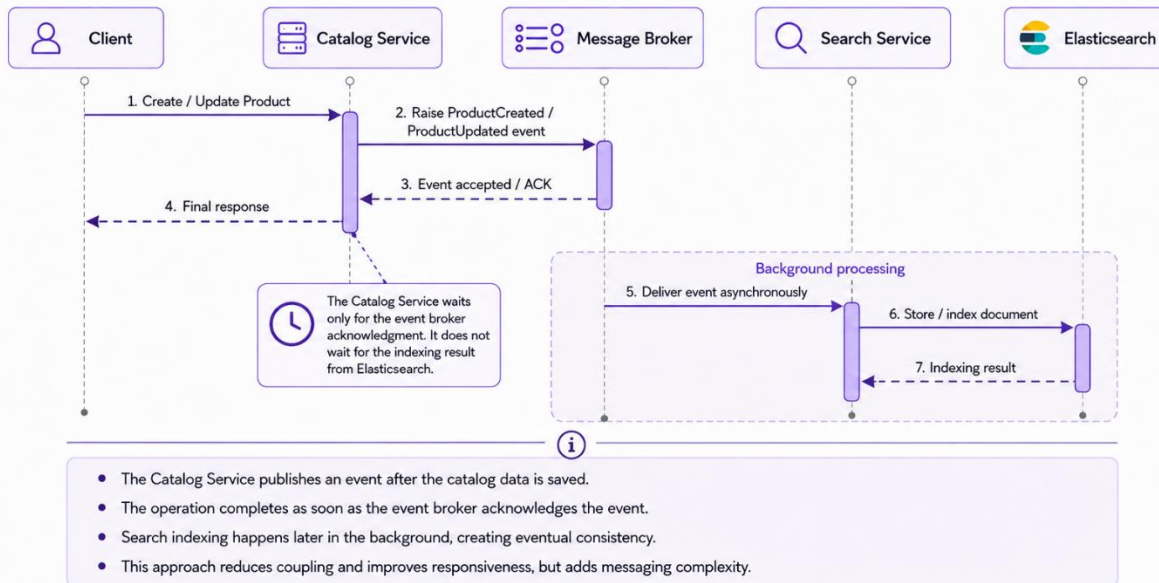
Synchronous Communication



Asynchronous Communication

Asynchronous Event-Driven Communication for Search Indexing

The caller waits only until the event is accepted, not until Elasticsearch indexing is completed.



For this scenario, we choose asynchronous communication. Messages can remain in the broker until a consumer is available, and transient processing failures can be retried according to the messaging configuration.

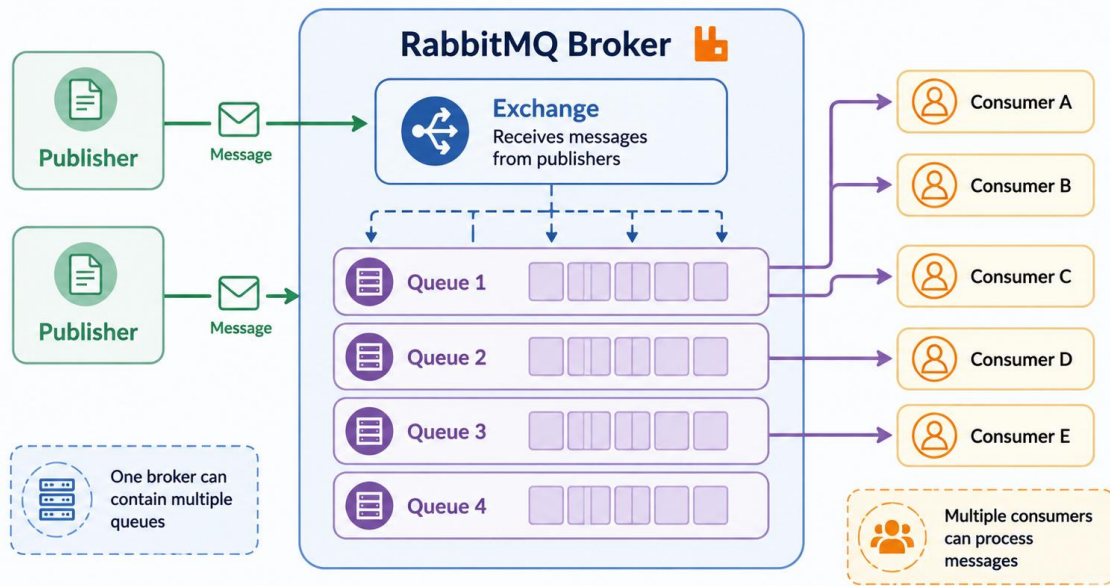
Additionally, the Catalog Service does not need to wait for the indexing operation to finish. Indexing can continue in the background, keeping the Catalog and Search services decoupled.

This approach is also useful when a large number of records must be indexed. We can run multiple instances of the Search Service as separate pods or processes. RabbitMQ distributes messages among competing consumers, allowing the workload to be processed in parallel. This increases throughput and can reduce the overall indexing time.

RabbitMQ

RabbitMQ is a message broker used to send, store, route, and deliver messages between services. It enables asynchronous communication and helps decouple services from each other.

RabbitMQ Messaging Flow



MassTransit

MassTransit is a .NET messaging framework that simplifies working with message brokers such as RabbitMQ. It provides abstractions for publishing, sending, consuming, retrying, and handling messages without dealing directly with the low-level RabbitMQ APIs.

At this point, the publisher side is complete and catalog data can be synchronized with Elasticsearch. The next step is to build the Search Service that consumes the integration events and queries the indexed data.

Search Service (Consumer)

We start with a blank solution and connect it to Elasticsearch by installing the official .NET client.

Reference: <https://www.elastic.co/docs/reference/elasticsearch/clients/dotnet/installation>

Before installing the NuGet package, check the client compatibility matrix and select a version that matches the Elasticsearch version used by the environment.

Client Version	Compatible with Elasticsearch 8.x	Compatible with Elasticsearch 9.x	Compatible with Elasticsearch 10.x
9.x	✗no	✓yes	✓yes
8.x	✓yes	✓yes	✗no

The client can be configured with the cluster URL, certificate fingerprint, and basic authentication credentials:

```
using Elastic.Clients.Elasticsearch;
using Elastic.Transport;
```

```
var settings = new ElasticsearchClientSettings(new Uri("https://localhost:9200"))
    .CertificateFingerprint("<FINGERPRINT>")
    .Authentication(new BasicAuthentication("<USERNAME>", "<PASSWORD>"));
```

```
var client = new ElasticsearchClient(settings);
```

The username and password come from the Elasticsearch configuration used by Docker Compose. To obtain the certificate fingerprint used by the .NET client, follow these steps:

- 1- Open Docker Desktop.
- 2- Select the `es01` container.
- 3- Open the Exec tab.
- 4- `cd /usr/share/elasticsearch/config/certs/ca`
- 5- `/usr/share/elasticsearch/jdk/bin/keytool -printcert -file ca.crt`
- 6- Copy the SHA-256 fingerprint.
- 7- Convert the fingerprint to the hexadecimal format expected by the client (for example, remove separators if required by the configuration).

Event Synchronization

The Search Service keeps its Elasticsearch index synchronized by consuming catalog integration events published through RabbitMQ.

```
CatalogItemAddedEvent
CatalogItemChangedEvent
CatalogItemDeletedEvent
```

Create and change events index the latest searchable representation of the item. The delete event removes the Elasticsearch document by using the catalog slug as the document identifier.

```
Catalog Service → RabbitMQ → Search Consumer → Elasticsearch
```

Health Checks

The Search Service exposes separate liveness and readiness endpoints. Liveness verifies that the API process can respond, while readiness also checks whether required dependencies such as Elasticsearch are available.

```
/health/live – Checks whether the Search API is running.
/health/ready – Checks whether the service is ready to handle requests, including dependency health.
```

Elasticsearch Search Endpoints

The final part of the Search Service is exposing different search capabilities through dedicated API endpoints.

Instead of putting every Elasticsearch feature into a single endpoint, the search functionality is divided into several endpoints. This keeps the main search lightweight and allows clients to use more advanced features only when they are required.

The Search Service currently provides the following endpoints:

```
/api/v1/search
/api/v1/search/suggestions
/api/v1/search/facets
/api/v1/search/highlight
```

Each endpoint has a specific responsibility.

1. Main Search Endpoint

The main endpoint is responsible for the standard catalog search operation.

```
GET /api/v1/search
```

It supports:

- Full-text search
- Multi-field search
- Field boosting
- Typo tolerance
- Brand filtering
- Category filtering
- Case-insensitive filtering
- Pagination
- Relevance-based result ordering

Multi-field Search

A catalog item contains several searchable fields:

Name

CatalogBrand

CatalogCategory

Description

Searching only one field would not provide a good e-commerce search experience.

For example, when the user searches for:

Samsung laptop

the relevant information may exist in the product name, brand, category, or description.

For this reason, the endpoint uses an Elasticsearch `MultiMatch` query.

```
.MultiMatch(mm => mm
  .Query(request.Query)
  .Fields(new[]
  {
    "name^4",
    "catalogBrand^3",
    "catalogCategory^2",
    "description"
  })
  .Fuzziness(new Fuzziness("AUTO"))
)
```

Field Boosting

Not all fields have the same importance.

A match in the product name is normally more relevant than the same match inside the description.

For this reason, different boost values are assigned to the searchable fields:

Field	Boost
Name	4
CatalogBrand	3
CatalogCategory	2
Description	1

For example, if the user searches for:

iPhone

a product such as:

Name: Apple iPhone 16 Pro

should rank higher than:

Name: USB-C Charger

Description: Compatible with iPhone devices

Both documents match the query, but the first result receives a higher relevance score because the match occurs in the Name field.

Fuzzy Search

Users frequently make typing mistakes while searching.

For example:

iphone -> ipone

samsung -> samsng

wireless -> wirless

To support these scenarios, automatic fuzziness is enabled:

```
.Fuzziness(new Fuzziness("AUTO"))
```

Elasticsearch automatically determines the allowed edit distance based on the length of the searched term.

This improves the search experience without requiring a separate spelling-correction system.

Search Filters

The endpoint also supports optional Brand and Category filters.

For example:

```
GET /api/v1/search?query=laptop&brand=Lenovo&category=Laptop
```

The query is implemented using an Elasticsearch `Bool` query:

`Bool`

```

├── Must
│   └── MultiMatch
└── Filter
    ├── Brand
    └── Category

```

The `MultiMatch` query is placed inside `Must` because it contributes to the relevance score.

Brand and Category are placed inside `Filter` because they only restrict the result set and should not affect relevance.

For example:

Search Query: laptop

Brand Filter: Lenovo

The search engine first finds products relevant to `laptop` and then limits the results to Lenovo products.

Case-insensitive Filtering

Brand and Category filters use their Elasticsearch `keyword` fields because filtering requires exact values.

```
.Field("catalogBrand.keyword")
and:
.Field("catalogCategory.keyword")
```

However, exact `Term` queries are normally case-sensitive.

Therefore, case-insensitive matching was enabled:

```
.CaseInsensitive(true)
```

As a result, all of the following values can match the same stored category:

Laptop

laptop

LAPTOP

LaPtOp

Pagination

Returning all search results in a single response is inefficient.

For this reason, the Search endpoint supports pagination through:

Page

PageSize

The Elasticsearch `From` value is calculated using:

```
var from = (request.Page - 1) * request.PageSize;
```

For example:

Page 1 -> From 0

Page 2 -> From 10

Page 3 -> From 20

```
when:
  PageSize = 10
```

The Elasticsearch query then uses:

```
.From(from)
.Size(request.PageSize)
```

The API response also contains pagination information:

Items

TotalCount

Page

PageSize

TotalPages

TotalPages is calculated as:

```
var totalPages = (int)Math.Ceiling(
    totalCount / (double)request.PageSize);
```

A maximum page size is also enforced to prevent clients from requesting an unnecessarily large number of documents in a single request.

2. Autocomplete Endpoint

Autocomplete is implemented as a separate endpoint:

```
GET /api/v1/search/suggestions
```

The purpose of this endpoint is different from the standard search operation.

The main Search endpoint answers:

Which products are relevant to this complete search query?

The Suggestions endpoint answers:

What products might the user currently be trying to type?

For example:

iph

may return:

Apple iPhone 16

Apple iPhone 16 Pro

Apple iPhone Case

The endpoint uses Elasticsearch `MatchBoolPrefix`.

```
.MatchBoolPrefix(m => m
    .Field(x => x.Name)
    .Query(request.Query)
)
```

This query analyzes the input and treats the final term as a prefix.

Conceptually:

apple iph

behaves approximately like:

apple AND iph*

This makes it suitable for search-as-you-type scenarios.

The suggestion response contains useful navigation information:

Name

CatalogCategory

CatalogBrand

Url

For example:

```
{
  "name": "Apple iPhone 16 Pro",
  "catalogCategory": "Mobile",
  "catalogBrand": "Apple",
  "url": "/catalog/api/v1/items/apple-iphone-16-pro"
}
```

The frontend can therefore display the suggestion and directly navigate to the selected product.

Autocomplete is intentionally separated from the main Search endpoint because prefix searching and full-text searching solve different problems.

3. Facets Endpoint

The Search Service also exposes a dedicated endpoint for facets:

```
GET /api/v1/search/facets
```

Facets are commonly used in e-commerce applications to build filter panels.

For example, a search for:

laptop

may return:

Brands

Lenovo	12
Asus	8
Acer	5

Categories

Laptop	18
Gaming	7

The endpoint uses Elasticsearch `Terms` aggregations.

Brand aggregation:

```
.Terms(t => t
  .Field("catalogBrand.keyword")
  .Size(request.Size)
)
```

Category aggregation:

```
.Terms(t => t
  .Field("catalogCategory.keyword")
```

```
.Size(request.Size)
)
```

The `.keyword` fields are used because aggregations need the complete field value instead of analyzed text tokens.

The aggregation result is converted into a simple API response:

```
public sealed record CatalogFacetItem(
    string Value,
    long Count);
```

For example:

```
{
  "brands": [
    {
      "value": "Lenovo",
      "count": 12
    }
  ],
  "categories": [
    {
      "value": "Asus",
      "count": 8
    }
  ],
  "categories": [
    {
      "value": "Laptop",
      "count": 15
    }
  ]
}
```

Since this endpoint only needs aggregation information and does not need catalog documents, the Elasticsearch request uses:

```
.Size(0)
```

This prevents Elasticsearch from returning unnecessary search documents.

4. Highlight Search Endpoint

Highlighting is also implemented through a dedicated endpoint:

```
GET /api/v1/search/highlight
```

Elasticsearch highlighting identifies exactly which parts of a document matched the user's query.

For example, when searching for:

iPhone

the response can contain:

Apple <mark>iPhone</mark> 16 Pro

Highlighting is configured for both the product name and description.

```
.Highlight(h => h
    .PreTags("<mark>")
    .PostTags("</mark>"))
```

```
.AddField(
    x => x.Name,
```

```
f => f.NumberOfFragments(0))

    .AddField(
      x => x.Description,
      f => f
        .FragmentSize(150)
        .NumberOfFragments(1))
  )
```

For the product name, the complete value is returned because names are normally short.

For the description, only a limited fragment is returned so that the API does not return unnecessarily large highlighted text.

The response keeps the original values and highlighted values separate:

Name

Description

HighlightedName

HighlightedDescription

For example:

```
{
  "name": "Apple iPhone 16 Pro",
  "description": "The latest Apple smartphone.",
  "highlightedName": "Apple <mark>iPhone</mark> 16 Pro",
  "highlightedDescription": null
}
```

The original data therefore remains unchanged, while the frontend can use the highlighted value when required.

Why Highlighting Is a Separate Endpoint

Highlighting requires Elasticsearch to perform additional processing after finding the matching documents.

It is useful for some user interfaces, but it is not required for every search request.

Therefore, it was intentionally separated from the main Search endpoint.

/search

-> lightweight standard search

/search/highlight

-> search with additional highlight processing

This keeps the standard search operation simpler and avoids unnecessary highlighting overhead when clients do not need it.

Current Search API Structure

The Search Service currently exposes the following search capabilities:

```

/api/v1/search
├── GET /
│   ├── MultiMatch
│   ├── Field Boosting
│   ├── AUTO Fuzziness
│   ├── Brand Filter
│   ├── Category Filter
│   └── Pagination
├── GET /suggestions
│   └── MatchBoolPrefix
├── GET /facets
│   └── Terms Aggregations
└── GET /highlight
    └── Search + Highlighting
  
```

This design separates different Elasticsearch capabilities according to their responsibilities.

The main Search endpoint remains focused on fast product retrieval, while additional features such as autocomplete, aggregations, and highlighting are available only when they are explicitly required.

This approach keeps the API easier to maintain and allows each search capability to evolve independently.

Implementation Snapshot

Capability	Implemented
Catalog add/change/delete synchronization	Yes
Multi-field search with field boosting	Yes
AUTO fuzziness	Yes
Brand and Category filters	Yes
Case-insensitive filtering	Yes
Pagination	Yes
Autocomplete / suggestions	Yes
Facets / terms aggregations	Yes
Highlighted search results	Yes
Liveness / readiness health checks	Yes
Sorting	Deferred

Future Improvements

The project is intentionally evolving in small increments. Planned improvements can include sorting, custom analyzers, synonyms, and semantic or hybrid search when those capabilities are needed.